Feature retrieval specification and usage in training and inference

This article describes the feature retrieval specification, and how to use a feature retrieval specification in training and inference.

A feature retrieval specification is an artifact that defines a list of features to use in model input. In a feature retrieval specification, the features:

- must exist in a feature set registered in a feature store
- can exist in multiple feature sets and multiple feature stores

The feature retrieval specification is used at the time of model training and at the time of model inference. These flow steps involve the specification:

1. Select features, and generate a feature retrieval specification
2. Use that specification and observation data to generate training data resource with a point-in-time join
3. Train a model with the generated training data.
4. Package the feature retrieval specification with the model artifact.
5. At model inference time, use the feature store SDK in the inference scoring script to load the feature retrieval specification from the model artifact folder, and look up features from the online store.

Create a feature retrieval specification

Use the feature store SDK to generate a feature retrieval specification. Users first select features, and then use the provided utility function to generate the specification.

Python

```
from azureml.featurestore import FeatureStoreClient
from azure.ai.ml.identity import AzureMLOnBehalfOfCredential

featurestore1 = FeatureStoreClient(
    credential=AzureMLOnBehalfOfCredential(),
```

```
    subscription_id=featurestore_subscription_id1,
    resource_group_name=featurestore_resource_group_name1,
    name=featurestore_name1,
)

features = featurestore1.resolve_feature_uri(
  [
    f"accounts:1:numPaymentRejects1dPerUser",
    f"transactions:1:transaction_amount_7d_avg",
  ]
)

featurestore2 = FeatureStoreClient(
    credential=AzureMLOnBehalfOfCredential(),
    subscription_id=featurestore_subscription_id2,
    resource_group_name=featurestore_resource_group_name2,
    name=featurestore_name2,
)

features.extend(
  featurestore2.resolve_feature_uri([
    f"loans:1:last_loan_amount",
  ])
)

featurestore1.generate_feature_retrieval_spec("./feature_retrieval_spec_folder",
features)
```

The **2.Experiment-train-models-using-features.ipynb** notebook, hosted at this resource , has detailed examples.

The function generates a YAML file artifact, which has a structure similar to the structure in this example:

YAML

```yaml
feature_stores:
  - uri: azureml://subscriptions/{sub}/resourcegroups/{rg}/workspaces/{featurestore-
workspace-name}
    location: eastus
    workspace_id: {featurestore-workspace-guid-id}
    features:
      - feature_name: numPaymentRejects1dPerUser
        feature_set: accounts:1
      - feature_name: transaction_amount_7d_avg
        feature_set: transactions:1
  - uri: azureml://subscriptions/{sub}/resourcegroups/{rg}/workspaces/{featurestore-
workspace-name}
```

```
    location: eastus2
    workspace_id: {featurestore-workspace-guid-id}
    features:
      - feature_name: last_loan_amount
        feature_set: loans:1
serialization_version: 2
```

Use feature retrieval specification to create training data

The feature store point-in-time join can create training data in two ways:

- The `get_offline_features()` API function in the feature store SDK in a Spark session/job
- The Azure Machine Learning built-in feature retrieval (pipeline) component

In the first option, the feature retrieval specification itself is optional because the user can provide the list of features on that API. However, if a feature retrieval specification is provided, the `resolve_feature_retrieval_spec()` function in the feature store SDK can load the list of features that the specification defined. That function then passes that list to the `get_offline_features()` API function.

Python

```python
from azureml.featurestore import FeatureStoreClient
from azureml.featurestore import get_offline_features
from azure.ai.ml.identity import AzureMLOnBehalfOfCredential

featurestore = FeatureStoreClient(
    credential=AzureMLOnBehalfOfCredential(),
    subscription_id=featurestore_subscription_id,
    resource_group_name=featurestore_resource_group_name,
    name=featurestore_name,
)

features =
featurestore.resolve_feature_retrieval_spec("./feature_retrieval_spec_folder")

training_df = get_offline_features(
    features=features,
    observation_data=observation_data_df,
    timestamp_column=obs_data_timestamp_column,
)
```

The second option sets the feature retrieval specification as an input to the built-in feature retrieval (pipeline) component. It combines that feature retrieval specification with other inputs - for example, the observation data set. It then submits an Azure Machine Learning pipeline (Spark) job, to generate the training data set as output. This option is recommended as a way to make the training pipeline ready for production, for repeated runs. Visit the feature retrieval component resource for more information about the built-in feature retrieval (pipeline) components.

Package a feature retrieval specification with model artifact

The feature retrieval specification must be packaged with the model artifact, in the root folder, when training a model on data with features from feature stores:

- Lineage tracking: For a model registered in an Azure Machine Learning workspace, the lineage between the model and the feature sets is tracked only if the feature retrieval specification exists in the model artifact. In the Azure Machine Learning workspace, the model detail page and the feature set detail page show the lineage.
- Model inference: At model inference time, before the scoring code can look up feature values from the online store, that code must load the feature list from the feature retrieval specification, located in the model artifact folder.

The feature retrieval specification must be placed under the root folder of the model artifact. Its file name can't be changed:

```
<model folder>/
├── model.pkl
├── other_folder/
│   ├── other_model_files
└── feature_retrieval_spec.yaml
```

The training job should handle the packaging of the specification.

If the built-in feature retrieval component generates the training data, the feature retrieval specification is already packaged with the training data set, under its root folder. This way, the training code can handle the copy, as shown here:

Python

```
import shutil

shutil.copy(os.path.join(args.training_data, "feature_retrieval_spec.yaml"),
args.model_output)
```

Review the **2.Experiment-train-models-using-features.ipynb** notebook, hosted at this resource , for a complete pipeline example that uses a built-in feature retrieval component to generate training data and run the training job with the packaging.

For training data generated by other methods, the feature retrieval specification can be passed as an input to the training job, and then handle the copy and package process in the training script.

Use feature retrieval specification in online inference

In the scoring script, the feature store SDK must load the feature retrieval specification before the online lookup is called. The scoring script `init()` function should handle the loading of the specification, as shown in this scoring script:

> **Note**
>
> Online feature lookup requires `azureml-featurestore` version 1.2.1 or later. For more information, see Troubleshooting managed feature store.

Python

```
from azure.identity import ManagedIdentityCredential
from azureml.featurestore import FeatureStoreClient
from azureml.featurestore import get_online_features, init_online_lookup

def init():
  credential = ManagedIdentityCredential()
  spec_path = os.path.join(os.getenv("AZUREML_MODEL_DIR"), "model_output")

  global features
  featurestore = FeatureStoreClient(credential=credential)
  features = featurestore.resolve_feature_retrieval_spec(spec_path)
  init_online_lookup(features, credential)
```

Visit the **4.Enable-online-store-run-inference.ipynb** notebook, hosted at this resource , for a detailed code snippet.

Use feature retrieval specification in batch inference

Batch inference requires:

- A feature retrieval specification, to generate the batch inference data
- A run of a model batch prediction on the inference data

The feature retrieval specification used in step 1 operates the same way as it does to generate training data. The built-in feature retrieval component generates the inference data. As long as the feature retrieval specification is packaged with the model, the model can serve, as a convenience, as the input to the component. This approach is an alternative to directly passing the inference data in the feature retrieval specification.

Visit the **3.Enable-recurrent-materialization-run-batch-inference.ipynb** notebook, hosted at this resource , for a detailed code snippet.

Built-in feature retrieval component

While the `azureml-featurestore` package `get_offline_feature()` function can handle feature retrieval in a Spark job, Azure Machine Learning offers a built-in pipeline component:

- The component predefines all the required packages and scripts to run the offline retrieval query, with a point-in-time join
- The component packages the feature retrieval specification with the generated output training data

An Azure Machine Learning pipeline job can use the component with the training and batch inference steps. It runs a Spark job to:

- retrieve feature values from feature stores (according to the feature retrieval specification)
- join, with a point-in-time join, the feature values to the observation data, to form training or batch inference data
- output the data with the feature retrieval specification

The component expects these inputs:

Expand table

Input	Type	Description	Supported value	Note
`input_model`	`custom_model`	Features from feature store train this model. The model artifact folder has a `feature_retrieval_spec.yaml` file that defines the feature dependency. This component uses the YAML file to retrieve corresponding features from the feature stores. A batch inference pipeline generally uses this component as a first step to prepare the batch inference data.	Azure Machine Learning model asset `azureml:<name>:<version>`, local path to the model folder, `abfss://` `wasbs://` or `azureml://` path to the model folder	only one of the `input_model` or the `feature_retrieval_spec` inputs is required
`feature_retrieval_spec`	`uri_folder`	The URI path to a folder. The folder must directly host a `feature_retrieval_spec.yaml` file. This component uses the YAML file to retrieve corresponding features from the feature stores. A training pipeline generally uses the corresponding feature retrieval as a first step to prepare the training data	Azure Machine Learning data asset `azureml:<name>:<version>`, local path to the folder, `abfss://` `wasbs://` or `azureml://` path to the folder	only one of the `input_model` or the `feature_retrieval_spec` inputs is required
`observation_data`	`uri_folder`	The observation data to which the features are joined	Azure Machine Learning data asset	

Input	Type	Description	Supported value	Note
			`azureml:<name>:<version>`, local path to the data folder, `abfss://` `wasbs://` or `azureml://` path to the data folder	
`observation_data_format`	`enum`	The feature retrieval job reads the observation data, according to the format	parquet, CSV, delta	
`timestamp_column`	`string`	Timestamp column name in the observation data. The point-in-time join uses the column on the observation data side		

The `output_data` is the only output component. The output data is a data asset of type `uri_folder`. The data always has a parquet format. The output folder has this folder structure:

```
<output folder name>/
├── data/
│ ├── xxxxx.parquet
│ └── xxxxx.parquet
└── feature_retrieval_spec.yaml
```

To use the component, reference its component ID in a pipeline job YAML file, or drag and drop the component in the pipeline designer to create the pipeline. This built-in retrieval component is published in the Azure Machine Learning registry. To find the latest version, check the Azure Machine Learning registry .

Review these notebooks for examples of the built-in component, both hosted at this resource :

- **2.Experiment-train-models-using-features.ipynb**

- **3.Enable-recurrent-materialization-run-batch-inference.ipynb**

Next steps

- Tutorial 1: Develop and register a feature set with managed feature store
- GitHub Sample Repository

Last updated on 01/28/2026